

CSCI 5607 Final Project Report

Fractal Flames

The goal of this project was to render some fractal flames with more basic variations such as linear and sinusoidal. The original goal was to do the normal, iterated system function fractals that appear more geometric. However, that changed when I came across fractal flames and found them more visually captivating. I was able to achieve what I set out to do and also added a spherical variation in there. One thing I didn't quite accomplish was to have enough iterations to populate a histogram grid so the variation(s) look more like a flame. The flames with my implementation are a bunch of somewhat sparse points at the moment, so it looks like a bunch of sparks. The general structure of the transformation and variation is still visible, such as the Barnsley fern transformation and a linear variation, but it doesn't look as nice and flame-like as more advanced implementations such as Apophysis.

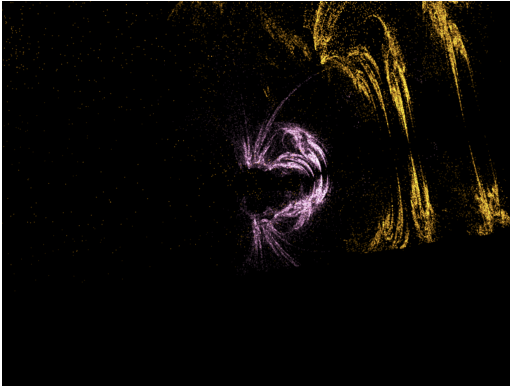
The main obstacles I encountered were learning the format of WebGL and getting a UI input, getting the affine coefficients for the transformation, and doing the chaos game with the 3D histograms. I was also minorly confused about resetting the animation frames (so the simulation doesn't speed up every time the user clicks the 'Regenerate' button) and WebGL worker logic for a little bit. For the chaos game, it was initially just difficult for me to understand the logic then applying it to the histogram structure, but after reading the Fractal Flame Algorithm paper a couple of times and some internet searching, it made more sense. Additionally, I did not implement the full process as mentioned in the paper, such as the post-transformations, final transforms, and motion blur. I also just used the normal gamma value of 2.2 and didn't really test out other values.

Connection to class: A good majority of the concepts from the fourth homework and project were implemented, such as shaders, data buffers, and OpenGL rendering, but with WebGL. It also involves mapping 3D coordinates to canvas width and height. Although I didn't play around with the brightness in this project, I still used the gamma factor. The brightness was adjusted based on how dense a histogram grid was by raising the calculated log density of the most dense (highest count) histogram square to the gamma factor. Animation was included also, but it was just rotating the fractal flame. All I did for that was consistently add a little bit to the rotation angle variable every frame then rotate the x and y so it looks continuous.

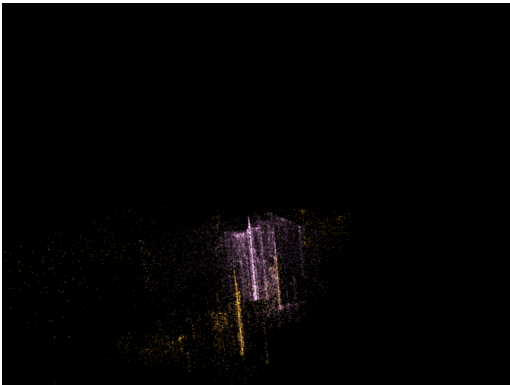
Key features: There is a simple UI control to be able to see which variations are being rendered. Since a random point in the bi-unit square is taken for the chaos game, the output will look different most of the time, even if the same variations may be checked. However, some are more predictable and consistent than others, such as the sinusoidal and spherical combination, where it's the same general shape every time but the points might be populated differently. The general flow of the program goes: variation(s) get selected, start flame generation, do the chaos game for

1 million iterations and 150,000 samples, calculate the log density of the histogram square with the highest count, then assign colors.

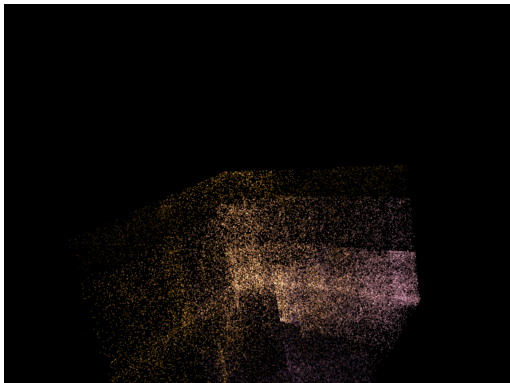
For the following images, the variations are represented as: {linear: 1, sinusoidal: 2, spherical: 3}. For example, a selection of the linear and sinusoidal variation can be represented as 2,3. All runs start with 'Default', which is only the linear variation.



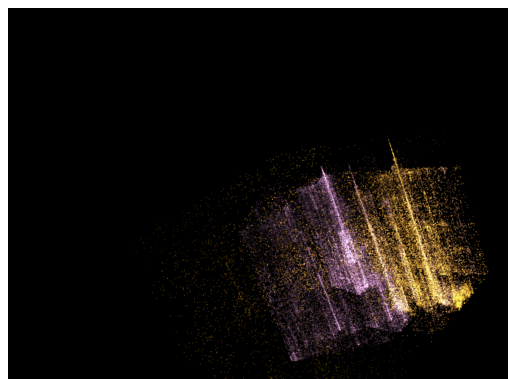
Default → 2,3



Default → 1,3 → 1,2



Default → 2. The regenerate button may have to be clicked multiple times to get this result.



Default → 2 → 1,2 (→ 1,2,3 optional). In this particular run, this result was achieved with only checking 1,2, but usually it also happens when I check 1,2,3.

Resources used:

Spotworks, Scott Draves and Erik Reckase Berthoud. "The Fractal Flame Algorithm." (2008).

https://flam3.com/flame_draves.pdf

<https://speice.io/2024/11/playing-with-fire/> (This one was really helpful in understanding the structure).

https://developer.mozilla.org/en-US/docs/Web/API/WebGL_API/Tutorial/Adding_2D_content_to_a_WebGL_context (This helped me get familiar with WebGL before trying to implement the flames).

When I search up a question, such as "how to make ui controls in javascript and html", I use the AI overview that always comes up. Sometimes I'll go to other website links below it too.

I used AI tools like Gemini to help me debug. About 80% of the time, it was a typo. Other times it gave me suggestions like "you should add `gl.blendFunc()`", which I didn't know existed but I do now. I also used it to make the learning process of the fractal flames go by faster, such as making Gemini summarize a section in the paper then I go back for the details, because initially, the paper was a bit difficult for me to understand.

Future work: I will definitely be adding more variations and transformations. I've been looking forward to trying to get the Julia set rendered. It would be cool to do more interactions and render more samples so the flames look more like flames instead of sparks. There also seems to be a box around how many points are being rendered, so I'm going to change that so it fills the canvas.

Additionally, I want to add more interactive features, such as the user being able to move around and rotate instead of having one continuous rotation, the user being able to determine the point of origin for a fractal flame with a mouse click or point selection, and if the user wants a continuous rotation, then being able to pause it.

I aspire for my fractal flames to look like those in Apophysis one day.